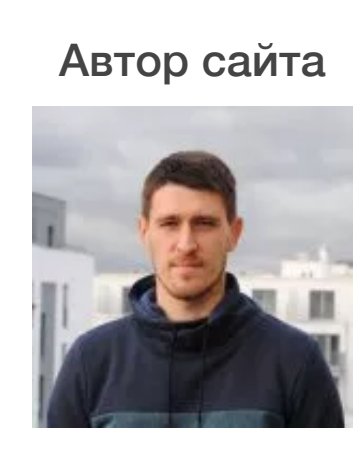
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс
Jetpack Compose

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 1. Введение. Что такое Dagger и зачем он нужен

22 марта 2021

В этом уроке я подробно расскажу о Dagger и его возможностях. Мы разберем, что такое Component и Module, подключим Dagger к проекту, и сделаем несколько простых примеров

*Если вас интересует тема **мультимодульности** и даггера, обратите внимание на эту серию статей.*

Зачем нужен Dagger

Если вы хотите снизить зависимость объектов друг от друга и упростить написание тестов для вашего кода, то вам подойдет паттерн Dependency Injection. А Dagger - это библиотека, которая поможет в реализации этого паттерна. В этом курсе я опишу использование библиотеки Dagger версии 2 (далее по тексту даггер).

Плюсы даггера в сравнении с другими библиотеками:
- генерирует код несложный для понимания и отладки
- проверяет зависимости на этапе компиляции
- не создает проблем при использовании proguard

Сразу скажу, что тема нетривиальная и у вас могут возникать вопросы типа "а что будет, если сделать так?". Рассмотреть все случаи в рамках этого курса я не смогу. Поэтому очень рекомендую вам создавать примеры и на них проверять, как все это работает в том или ином случае. Это поможет вам лучше понять теорию.

Пример

Чтобы понять, зачем нам может понадобиться Dependency Injection и даггер, давайте рассмотрим небольшой пример. В нем мы смоделируем ситуацию, когда создание одного объекта может потребовать создание еще нескольких.

Пусть в нашем приложении есть MainActivity и, в соответствии с паттерном MVP, для него есть презентер. Презентеру для работы нужны будут некие UserController и DataController. Т.е. нам надо будет создать два этих объекта перед тем, как создать презентер. Но для создания двух этих объектов нам, в свою очередь, нужны объекты ApiService и SharedPreferences. А для создания ApiService нужны RestAdapter, RestAdapter.Builder, OkHttpClient и Cache.

В обычной реализации это может выглядеть так:

```
class MainActivity : AppCompatActivity() {

    lateinit var presenter: MainActivityPresenter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val cacheDirectory = File("some path")
        val cache: Cache = HttpResponseCache(cacheDirectory, 50 * 1024 * 1024)

        val httpClient = OkHttpClient()
        httpClient.setCache(cache)

        val builder = RestAdapter.Builder()
        builder.setClient(httpClient)
        val restAdapter: RestAdapter = builder.build()
        val apiService: ApiService = restAdapter.create(ApiService::class.java)

        val userController = UserController(apiService)

        val preference = getSharedPreferences("user_prefs", MODE_PRIVATE)
        val dataController = DataController(preference)

        activityPresenter = MainActivityPresenter(this, userController, dataController)
    }
}
```

В MainActivity мы создаем один объект, затем используем его при создании другого, и так далее по цепочке, чтобы в итоге получить презентер. Нам сейчас не важно, какие именно объекты создаются. Главное - это сколько кода может потребоваться написать в MainActivity, чтобы получить результат.

Выглядит это все не очень красиво, а главное - неправильно. Activity ничего не должно знать о кэшах, сервисах, контроллерах и прочем. Activity знает только презентер и должно получать его в готовом виде.

Как вариант - можно весь этот создающий код поместить в сам презентер. Т.е. Activity только создает презентер, а он уже пусть внутри себя создает все остальные объекты. Так сделать можно, но это тоже неправильно, потому что нарушает принцип Dependency Injection: презентер не должен создавать эти объекты, он их должен получать готовыми. Да и написать нормальные тесты для такого презентера будет затруднительно.

Нам нужен какой-то механизм, который умеет сам создавать все необходимые объекты, как мы это делали в коде Activity. Т.е. по цепочке, создает один объект, использует его в конструкторе другого объекта и так далее, пока не получится готовый презентер, который будет торжественно вручен Activity.

Даггер как раз является таким механизмом. При его использовании код в Activity будет выглядеть так:

```
class MainActivity : AppCompatActivity() {

    lateinit var presenter: MainActivityPresenter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        presenter = (application as App).appComponent.getMainActivityPresenter()

    }
}
```

Разумеется, код создания объектов никуда не исчез. Но он разделен на части и вынесен из Activity в специальные отдельные классы, к которым даггер имеет доступ. В итоге мы просто вызываем метод getMainActivityPresenter, чтобы получить презентер. А даггер под капотом уже сам создаст этот объект и всю необходимую для него иерархию объектов.

Теория

Теперь давайте смотреть, как работает даггер изнутри.

Возьмем все тот же пример с Activity и Presenter. Т.е. когда Activity для своих нужд создает объект Presenter.

Обычная схема создания будет выглядеть так:

Activity > Presenter

Т.е. Activity создает Presenter самостоятельно

При использовании даггера схема будет выглядеть так:

Activity -> Component -> Module -> Presenter

Activity обращается к компоненту (appComponent в примере выше), компонент с помощью модулей создает Presenter (и все остальные необходимые для этого объекты) и возвращает его в Activity.

Модули и компоненты - это два ключевых понятия даггера.

Модули - это просто классы, в которые мы выносим (из Activity) код создания объектов. Обычно каждый модуль включает в себя создание объектов близких по смыслу.

Например:

Модуль UserModule будет содержать в себе код создания объектов, связанных с пользователями, т.е. UserController.

Модуль NetworkModule - объекты OkHttpClient и ApiService.

Модуль StorageModule - объекты DataController и SharedPreferences

Компонент - это посредник между Activity и модулями. Когда Activity нужен какой-либо объект, она сообщает об этом компоненту. Компонент знает, какой модуль умеет создавать такой объект, просит модуль создать объект, и передает его в Activity. При этом компонент может использовать другие модули, чтобы создать всю иерархию объектов, необходимую для создания искомого объекта.

Процесс работы даггера можно сравнить с обедом в McDonalds. Т.е. по аналогии со схемой даггера:

Activity -> Component -> Module -> Presenter

схема McDonalds выглядит так:

Клиент -> Кассир -> Производственная линия -> Заказ (Бигмак/Картошка/Кола)

Рассмотрим подробнее шаги этих схем:

McDonalds	Даггер
Клиент определился, что его заказ будет состоять из бигмака, картошки и колы, и он говорит об этом кассиру	Activity сообщает компоненту, что ему понадобится Presenter
Кассир ходит по производственной линии и собирает заказ: берет бигмак, наливает колу, насыпает картошку	Компонент использует модули, чтобы создать все необходимые объекты, которые понадобятся для создания Presenter
Кассир комплектует заказ в пакет или на поднос и выдает его клиенту	Компонент в итоге получает от модулей требуемый объект Presenter и отдает его Activity

Практика

Теперь на простом примере посмотрим, как создавать модули и компоненты, и как с их помощью Activity будет получать требуемые объекты.

В этом курсе все примеры будут на Kotlin.

Подключение

Для начала подключите плагин kapt. Это делается в самом начале файла build.gradle модуля

Подключить можно так:

```
plugins {
    ...
    id 'kotlin-kapt'
}
```

или так:

```
apply plugin: 'kotlin-kapt'
```

Добавьте в раздел dependencies файла build.gradle модуля строки:

```
implementation 'com.google.dagger:dagger:2.34'
kapt 'com.google.dagger:dagger-compiler:2.34'
```

Объекты

В качестве объектов, которые мы будем запрашивать от даггера, используем пару классов: DatabaseHelper и NetworkUtils.

```
class DatabaseHelper {

}
```

```
class NetworkUtils {

}
```

Их реализация нам сейчас не важна, оставляем их пустыми.

Предположим, что эти объекты будут нужны нам в MainActivity.

```
class MainActivity : AppCompatActivity() {

    lateinit var databaseHelper: DatabaseHelper
    lateinit var networkUtils: NetworkUtils

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // ...
    }
}
```

Презентер пока не используем, чтобы не усложнять пример.

Чтобы получить эти объекты с помощью даггера, нам нужно создать модули и компонент.

Модули

Создаем модули, которые будут уметь предоставлять требуемые объекты. Именно в модулях мы и пишем весь код по созданию объектов. Это обычные классы, но с парой аннотаций.

Модуль для DatabaseHelper:

```
@Module
class StorageModule {

    @Provides
    fun provideDatabaseHelper(): DatabaseHelper {
        return DatabaseHelper()
    }

}
```

Модуль для NetworkUtils:

```
@Module
class NetworkModule {

    @Provides
    fun provideNetworkUtils(): NetworkUtils {
        return NetworkUtils()
    }

}
```

Аннотацией @Module мы сообщаем даггеру, что этот класс является модулем. А аннотация @Provides указывает, что метод является поставщиком объекта.

Таким образом мы предоставляем даггеру информацию о том, как создавать объекты DatabaseHelper и NetworkUtils. Нам больше не надо будет самим создавать их. Теперь, где бы они нам не понадобились, мы сможем просто попросить эти объекты у даггера, и он создаст их и предоставит нам.

Технически можно было вполне обойтись и одним модулем. Но логичнее будет разделить объекты на модули по их смыслу и области применения.

Компонент

Модули готовы, теперь создаем компонент. Для этого нам необходимо создать интерфейс:

```
@Component
interface AppComponent {

}
```

Данный интерфейс описывает пустой компонент, который пока ничего не умеет.

При компиляции проекта, даггер найдет этот интерфейс по аннотации @Component и сгенерирует класс DaggerAppComponent (Dagger + имя интерфейса), который станет реализацией этого интерфейса. Этот класс и будет потом создавать объекты и возвращать их нам.

После того как даггер создал нам класс компонента, нам необходимо создать экземпляр этого класса. Это можно сделать, например, в Application классе (не забудьте добавить его в манифест):

```
class App: Application() {

    lateinit var appComponent: AppComponent

    override fun onCreate() {
        super.onCreate()
        appComponent = DaggerAppComponent.create()
    }
}
```

У класса компонента есть метод create, которым мы создаем его экземпляр.

Можно, кстати, сделать чуть короче:

```
class App: Application() {

    val appComponent = DaggerAppComponent.create()

}
```

На этом месте ваша среда разработки, возможно, будет ругаться на класс DaggerAppComponent. Так может происходить, потому что класса DaggerAppComponent пока не существует. Мы описали интерфейс компонента AppComponent, но нам надо скомпилировать проект, чтобы даггер создал этот класс-компонент.

Скомпилируйте проект. В Android Studio это можно сделать через меню Build -> Make Project (CTRL+F9). После того как процесс завершится, класс DaggerAppComponent будет создан в недрах папки build\generated\. Студия теперь знает этот класс и должна предлагать добавить его в import, чтобы в коде не было никаких ошибок.

Чтобы добраться до компонента из MainActivity, мы можем сделать так:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val appComponent = (application as App).appComponent

    // ...

}
```

После этого MainActivity сможет получать от компонента все необходимые объекты.

Нам остается рассказать компоненту, какие именно объекты мы хотим от него получать. Для этого мы будем наполнять его интерфейс методами. И вот тут у нас есть два вида методов.

Первый - это обычный и понятный get метод:

```
@Component(...)
interface AppComponent {
    fun getDatabaseHelper(): DatabaseHelper
}
```

Т.е. мы просто описываем метод, который должен вернуть нужный нам объект. Когда даггер будет создавать класс-реализацию (DaggerAppComponent) этого интерфейса, он создаст в нем и реализацию метода getDatabaseHelper. Этот метод будет ходить в модуль StorageModule, создавать объект DatabaseHelper и возвращать его нам.

Второй вид методов посложнее для понимания. Это inject метод.

```
@Component(...)
interface AppComponent {
    fun injectMainActivity(mainActivity: MainActivity)
}
```

В этом случае мы не просим компонент создать и вернуть нам конкретный объект. Вместо этого мы просим компонент проверить какие объекты нужны в MainActivity. Компонент создаст эти объекты и передаст их сразу в MainActivity. Такая процедура называется инджект.

Рассмотрим оба способа подробнее.

Get метод

Нам от компонента нужны объекты DatabaseHelper и NetworkUtils. Для этого нам надо просто добавить в интерфейс компонента методы, которые будут возвращать эти объекты:

```
@Component(modules = [StorageModule::class, NetworkModule::class])
interface AppComponent {
    fun getDatabaseHelper(): DatabaseHelper
    fun getNetworkUtils(): NetworkUtils
}
```

Создаем два метода. Они могут быть с любым именем, главное - это их возвращаемые типы (NetworkUtils и DatabaseHelper). Они дают понять компоненту, какие именно объекты мы хотим от него получать. При компиляции, даггер проверит, в каком модуле какой объект можно достать, и нагенерит в реализации двух этих методов соответствующий код создания этих объектов.

Список modules - это модули. Здесь нам надо указать модули, в которых компонент сможет найти код создания объектов. В StorageModule он найдет код для создания DatabaseHelper, а в NetworkModule - для NetworkUtils.

В MainActivity мы просто вызываем эти методы компонента, чтобы получить готовые объекты:

```
class MainActivity : AppCompatActivity() {

    lateinit var databaseHelper: DatabaseHelper
    lateinit var networkUtils: NetworkUtils

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val appComponent = (application as App).appComponent

        databaseHelper = appComponent.getDatabaseHelper()
        networkUtils = appComponent.getNetworkUtils()
    }
}
```

Если этот код у вас крэшит с NullPointerException, убедитесь, что добавили App класс в манифест.

Inject метод

У нас в MainActivity сейчас всего два объекта, которые мы получаем от компонента. Но если их будет штук 20, то придется в интерфейсе компонента описать 20 get-методов и в коде MainActivity написать 20 вызовов этих методов. У даггера есть более удобное решение для таких случаев. Мы можем научить компонент не возвращать объекты, а самому наполнять Activity требуемыми объектами. Т.е. мы даем компоненту экземпляр MainActivity, а он смотрит, какие объекты нужны, создает их и сам помещает в соответствующие поля.

Перепишем интерфейс компонента

```
@Component(modules = [StorageModule::class, NetworkModule::class])
interface AppComponent {
    fun injectMainActivity(mainActivity: MainActivity)
}
```

Вместо пары get-методов мы описываем один inject-метод. Имя может быть любым, главное - это тип его единственного параметра. Мы указываем здесь MainActivity. Тем самым, мы говорим компоненту, что когда мы будем вызывать этот метод и передавать туда экземпляр MainActivity, мы ожидаем, что компонент наполнит этот экземпляр требуемыми объектами.

Перепишем MainActivity:

```
class MainActivity : AppCompatActivity() {

    @Inject lateinit var databaseHelper: DatabaseHelper
    @Inject lateinit var networkUtils: NetworkUtils

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        (application as App).appComponent.injectMainActivity(this)
    }
}
```

Аннотациями @Inject мы помечаем поля, которые компонент должен заполнить (инджектить). При вызове метода injectMainActivity компонент создаст объекты DatabaseHelper и NetworkUtils и поместит их в соответствующие поля MainActivity.

Этот механизм можно посмотреть в коде класса (DaggerAppComponent) компонента, который был сгенерирован даггером. Метод injectMainActivity, если поубирать все лишнее, выглядит примерно так:

```
@Override
public void injectMainActivity(MainActivity instance) {
  instance.databaseHelper = databaseHelperProvider.get();
  instance.networkUtils = networkUtilsProvider.get();
}
```

Разумеется, get-методы и inject-методы могут быть использованы вместе в одном компоненте. Я описывал их отдельно друг от друга только для простоты понимания.

Где и как размещать все эти классы и интерфейсы зависит от вашей архитектуры и является отдельной темой для дискуссии.

Вперёд

Комментарии

# **Андрей** — **Андрей** 08.11.2021 18:08
Для корректного подключения Dagger к Kotlin проекту в 2021 году в build.gradle нужно добавить:

в секцию plugins:
id 'kotlin-kapt'

в dependencies:
implementation 'com.google.dagger:dagger:2.35.1'
kapt 'com.google.dagger:dagger-compiler:2.35.1'

Ответить

# **RE: Андрей** — **Алекс** 20.12.2022 18:20
для котлина 1.7
implementation 'com.google.dagger:dagger:2.44.2'
kapt 'com.google.dagger:dagger-compiler:2.44.2'

Ответить

# **RE: Урок 1. Введение** — **Azat** 29.01.2022 06:25
val appComponent = DaggerAppComponent.create()

для того, чтобы найти DaggerAppComponent нужно сначало нажать Run

Ответить

# **RE: Урок 1. Введение** — **Юрий** 12.02.2022 12:46
Достаточно нажать "make project"

Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.